

COMP2048: Theory of Computing

Dr. Kasra Khosoussi

Week 1

26/2/2026

Welcome to COMP2048: Theory of Computing

What we'll do today before the technical topics:

- Big picture + why it matters
- Course aims & learning outcomes
- How the course runs + assessments + expectations

Fundamental Questions We Study in COMP2048

- How to *define* computation?
- How do we *formalize* and *prove* claims about computation?
- What are the essential elements of computing?
- What problems *cannot* be solved by any algorithm?
- What problems are solvable *efficiently* with limited resources?
(e.g., memory, time)
- What's all the fuss about quantum computing?

Why This Course Matters

- Foundations of Computer Science
- Solid and rigorous foundation for your Algorithms courses
- Understanding what computers *cannot* do
- Understanding efficiency limits (complexity, P vs NP)
- Broader view of computation (e.g., quantum models)
- Skill you'll take anywhere: *precise thinking & reasoning*

Learning Outcomes

After successfully completing this course you should be able to:

- Define core concepts: computation, algorithms, decision problems, formal languages
- Study models of computation:
Finite Automata, Pushdown Aut. , Turing Machines, Cellular Aut.
- Analyze expressive power + limitations of computational models
- Apply proof techniques for correctness, equivalence, and impossibility results
- Determine decidability/undecidability using appropriate tools
- Explain complexity in terms of time/space resources
- Classify problems in basic complexity classes (incl. P and NP)
- Describe principles of quantum computation

Prerequisites & Assumed Background

- (MATH1061 or MATH1081) and (CSSE1001 or ENGG1001)
- Assumed background:
 - Basic Python coding
 - Discrete math basics: sets, functions/relations, graphs, Boolean ops
 - Be comfortable with reading and/or writing mathematical arguments & proofs

Teaching Team

- Course Coordinator (and lecturers):

Dr. Kasra Khosoussi & Dr. Josh Arnold

- Guest lectures by Dr. Riddhi Gupta
- Excellent team of tutors!
(too many to list – you will meet most of them soon)

How the Course Runs

- Lectures: Weeks 1-13
- Applied classes: **Weeks 2-13**
- Practical classes: **Weeks 10-13**

What to expect:

- Lectures: main material, core concepts
- Applied classes:
 - Problems from topics covered in previous week's lecture
 - Guided problem solving
 - Q&A and assignment support
- Practical classes: support for Assignment 3

Lectures Outline

- W1-3 Basics, Finite Aut. and Regular Languages
- W4 (Josh) Pushdown Aut. & Context-free Grammars/Languages
- W5-6 Turing Machines and Decidability
- W7 (Josh) Cellular Automata
- W8-9 Intro to Computational Complexity Theory (e.g., P vs. NP)
- W10-12 (Riddhi) Quantum Computing
- W13 (Josh) Other models of computation
- **Plan:** Applied class on week n will cover the topics presented in week $n - 1$'s lecture

Assessment at a Glance

- Assignment 1 (from topics covered in Weeks 1–3) problem set 20%
- Assignment 2 (from topics covered in Weeks 4–7) problem set 20%
- Assignment 3 (from topics covered in Weeks 8–12) problem set + some coding 20%
- Final Exam (written, in-person, closed-book) 40%
- All assessments (A1-A3 & Final Exam) are subject to **hurdles**
- Must satisfy **minimum requirements** to earn certain **grades/pass**

see course profile for details

Key Dates (Current Schedule)

- Assignment 1:
will be posted in Week 2 – due on **Fri 27 Mar 2026, 4:00pm**
- Assignment 2:
will be posted in Week 5 – due on **Fri 1 May 2026, 4:00pm**
- Assignment 3:
will be posted in Week 9 – due on **Fri 29 May 2026, 4:00pm**
- Submission: online by due date/time (via Blackboard/Gradescope)
- Late penalty: please see the course profile.
- Extensions:
 - Apply via the official process (see the course profile)
 - Maximum extension typically **3 days**
- Start early! Some problems may take longer than you think.
- Submit 1 hour before the deadline to be safe!

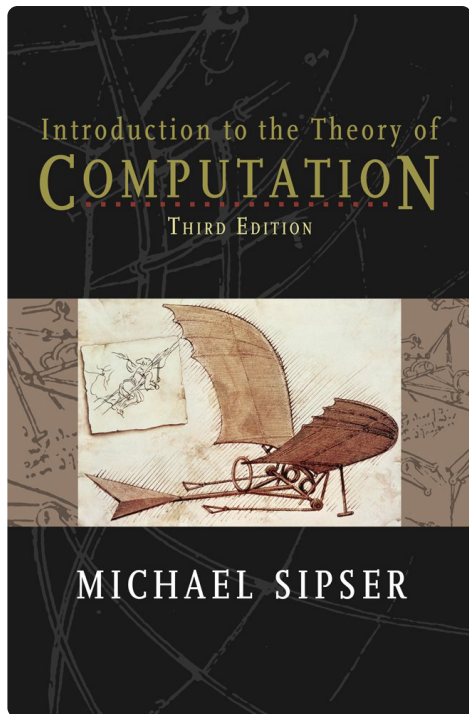
see course profile for details

Academic Integrity + Generative AI

- You are responsible for the work you submit.
- If you use generative AI:
 - **You must acknowledge/reference that use** (e.g., prompts you used)
 - You must be able to **explain and defend** your submission
 - Staff may conduct random interviews to verify understanding
- **Advice:** Use these tools to *support your learning*, not to replace your thinking. Attempt all assignment problems on your own first. After you have written your solutions, use AI to explore further, check your understanding, or deepen your insights.
- The assignments are meant to prepare you for the final exam (secured, closed-book).

see course profile for details

Learning Resources



How to Succeed

- The final exam questions will be similar in level of difficulty (though not identical) to problems from your assignments and applied classes.
- The assignments and applied class problem sets are designed to prepare you for the final exam.
- Use AI responsibly (see the slide on AI).
- Ask for help early.
- Attendance is not mandatory or formally recorded. However, regularly attending lectures (or, at least, watching the recordings) and actively participating in applied classes (and Ed) will significantly support your understanding and preparation for the assessments.
- Review the hurdle requirements on the course profile.

Friendly Requests :)

- **Please ask questions!**
(e.g., could you repeat X? I don't understand Y? etc)
- **Please bring your ideas and join the discussion! Don't be shy :)**
- Please be respectful and collegial (I know you will be!) and help each other learn (e.g., on Ed, study groups, etc).
- We are creating new content and restructuring the course this semester — thank you in advance for your patience.
- Our teaching team is working really hard to provide you with a great learning experience. Mistakes may/will happen, but we will fix them. Please let us know if you spot one.

Where to Find Stuff

- **Course Profile:** due dates, hurdles, policies, etc
- **Blackboard:** announcements, weekly materials, recordings, assignments, submission (via Gradescope), applied class worksheet, grades, etc
- **Ed Discussion:** Q&A, announcements, etc

Please regularly check:

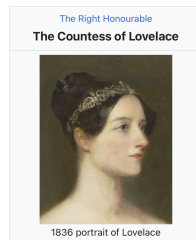
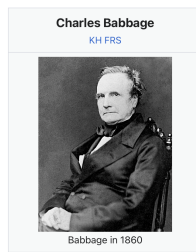
- Email (announcements/notifications/etc)
- Blackboard
- Ed Discussion – we may have already answered your question :)

Today's Plan

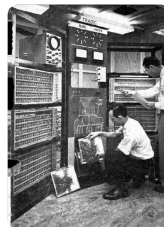
- Brief overview of (recent) history of computers and computing
- Working definition of computing
- Computational problems
- Overview of models of computation
- Deterministic Finite Automata (DFA, a.k.a. Finite State Machines)

Historical Context

- Analytical Engine (1800s)
- ENIAC (1940s) – vacuum tubes
- Transistor era (1950s–60s) led to smaller, faster machines
- Personal Computers (1970s)
- Today: ∞



computerhistory.org/timeline/computers/



First mention of “computer” in the New York Times

The first mention of “computer” in the
New York Times ✕ Post

A COMPUTER WANTED.

WASHINGTON, May 1.—A civil service examination will be held May 18 in Washington, and, if necessary, in other cities, to secure eligibles for the position of computer in the Nautical Almanac Office, where two vacancies exist—one at \$1,000, the other at \$1,400..

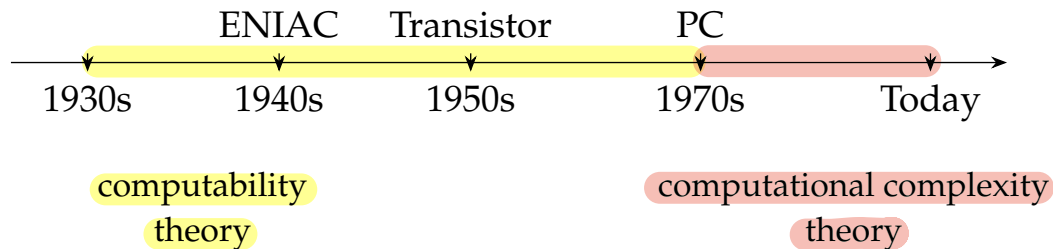
The examination will include the subjects of algebra, geometry, trigonometry, and astronomy. Application blanks may be obtained of the United States Civil Service Commission.

The New York Times

The first time “Computer” has appeared in the New York Times? (1892)



Timeline



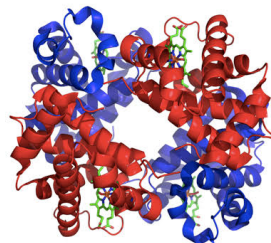
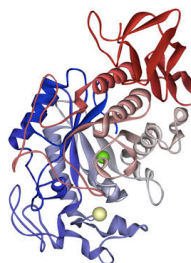
What is Computing?

- **Computing** is the process of taking inputs and applying a well-defined procedure (algorithm) to produce outputs.
- “Algorithm” derived from Al-Khwarizmi (c. 800 AD).
- Central to many scientific fields, e.g.,
 - Nobel prizes in Physics and Chemistry this year!
 - Biology (drug discovery, etc)

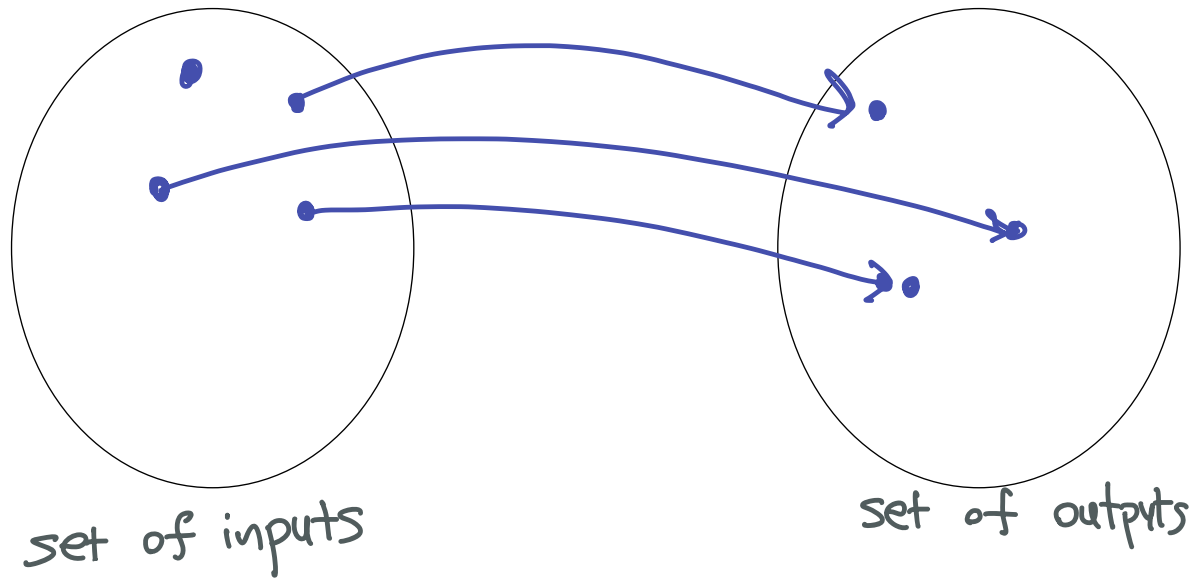
What is Computing?

- **Computing** is the process of taking inputs and applying a well-defined procedure (algorithm) to produce outputs.
- “Algorithm” derived from Al-Khwarizmi (c. 800 AD).
- Central to many scientific fields, e.g.,
 - Nobel prizes in Physics and Chemistry ~~this year!~~
 - Biology (drug discovery, etc)

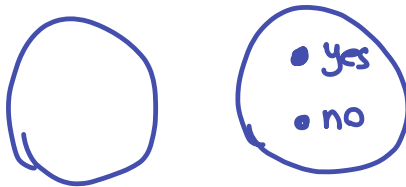
last



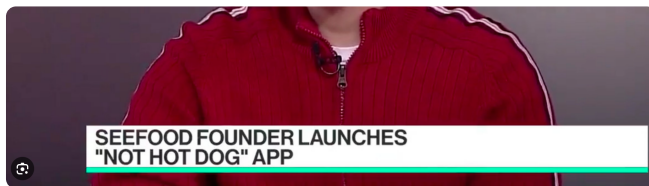
Computational Problem



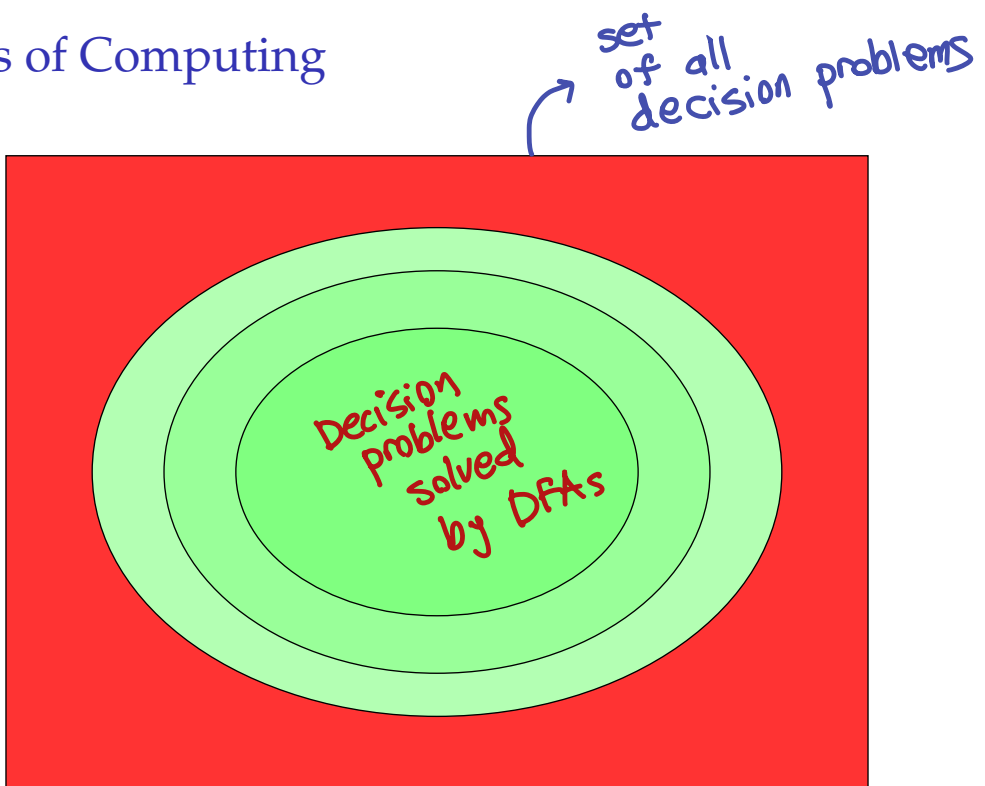
Decision Problems



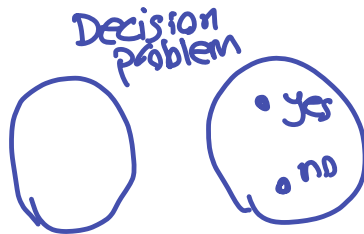
- A computing problem can be framed as a function from inputs to outputs.
- **Decision problem:** output is simply Yes or No (or 1 or 0; accept or reject).
- Image classification example: dog or cat? (or, cat or not-cat).



Models of Computing



Symbols, Strings, and Alphabets



- Inputs often modeled as **strings** over an alphabet Σ .
- Alphabet Σ is a finite (non-empty) set of symbols.
- Example:

$\Sigma = \{0,1\}$ (binary)

↓

10100
string

$\Sigma = \{a,b,c,\dots,z\}$ (lower-case English letters)

↓

abcdefghijklmnopqrstuvwxyz

Strings and Kleene Star

②



- A string x is a finite sequence of symbols. **Input**
- String length $|x|$ = number of symbols.
- Empty string ϵ has length 0.
- Can encode text, images, numbers as **binary strings** (though encodings vary).
- Kleene star Σ^* : set of all (finite) strings from alphabet Σ .

$$\Sigma = \{0,1\} \Rightarrow \Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$$

Languages and Decision Problems

- **Decision problem:** Does input x satisfy property P ?

- Model as a language $L \subseteq \Sigma^*$:

- If $x \in L \Leftrightarrow$ answer = yes (accept)
- If $x \notin L \Leftrightarrow$ answer = no (reject)

- Examples:

- All binary strings with an even number of 1s.
- All strings that start with 1 and end with 0.
- Strings matching the pattern: $(ab)^n$ for some $n \geq 0$.

$\{10, 100, 11000, \dots\}$

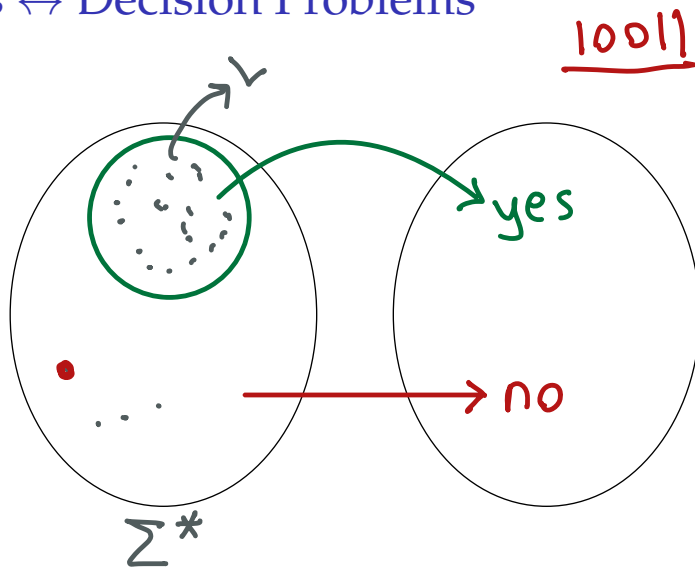
- Each **decision problem** corresponds to the **set of strings that are accepted** (the “yes/accept” set).

↗ string

$$L \subseteq \Sigma^*$$

$\{\epsilon, 0, 00, 000, \dots\}$
 $101, \dots$

Languages $\leftrightarrow$ Decision Problems



Therefore:

solving a decision prob. $\Leftrightarrow$ recognizing the corresponding language¹

¹i.e., given an input $w \in \Sigma^*$, decide whether $w \in L$ or $w \notin L$

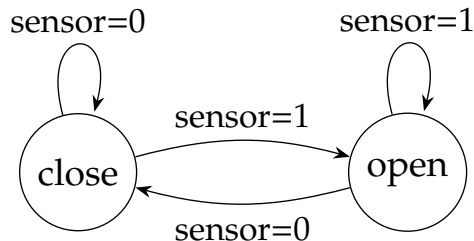
Deterministic Finite Automata

State Machines (Informal)

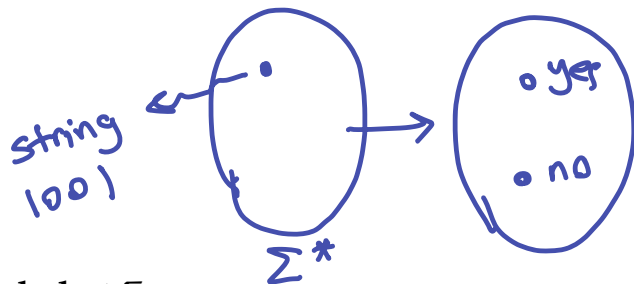
Example: automatic door

- **Finite number of states**
- State transition rules
- State changes each time based on:
 - **current state**
 - **current input symbol**
(e.g., a sensor reading)
- At every step:

(current state, input symbol) $\longrightarrow$ next state



From Doors to DFA

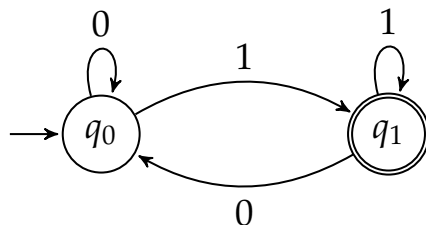


- **Input:** A string over an alphabet Σ
- The machine reads symbols **left to right**, one at a time
- After reading the last symbol, the machine must decide:

Accept or Reject the string?

First DFA Example

- Alphabet $\Sigma = \{0,1\}$
- States: q_0 and q_1
- Initial state: q_0
- Accept state(s): q_1



Tracing input strings:

- $w = 100$:

$$q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{0} q_0$$

ends in q_0 which is not an accepting state $\Rightarrow w = 100$ is rejected!

- $w = 11$:

$$q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_1$$

ends in q_1 which is an accepting state $\Rightarrow w = 11$ is accepted!

- $w = \varepsilon$:

q_0

ends in q_0 which is an accepting state $\Rightarrow w = \varepsilon$ is rejected!

✓ not

How DFA Process Input Strings

- Start in the initial state.
- For each symbol a in the input string:

$$\text{current state} \leftarrow \delta(\text{current state}, a)$$

- After reading the entire string:
 - If the current state is in F (i.e., set of accepting states) $\rightarrow$ **accept**
 - Otherwise $\rightarrow$ **reject**

Deterministic Finite Automata (DFA)

A DFA consists of:

- A finite set of states
- An input alphabet Σ
- A start state
- A set of accepting states
- A transition rule:

(current state, input symbol) $\rightarrow$ next state

Deterministic

For every state and every input symbol, there is **exactly one** next state.

Deterministic Finite Automata (DFA)

A DFA consists of:

- A finite set of states
- An input alphabet Σ
- A start state
- A set of accepting states
- A transition rule:

(current state, input symbol) $\rightarrow$ next state

Deterministic

For every state and every input symbol, there is **exactly one** next state.

Formal Definition of a DFA

A DFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states.
- Σ is a finite input alphabet.
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function.
- $q_0 \in Q$ is the start state.
- $F \subseteq Q$ is the set of accepting states.

Understanding the Transition Function

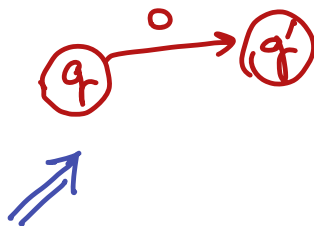
$$\delta : Q \times \Sigma \rightarrow Q$$

- Input: a pair (state, symbol)
- Output: a state
- Example:

$$\delta(q, 0) = q'$$

means:

- If we are in state q
- and read symbol 0
- then we move to state q' .



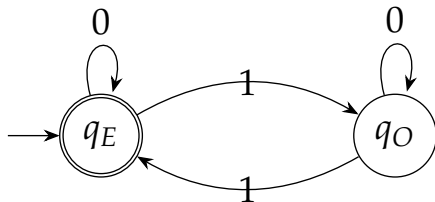
Example: Even Number of 1's

$\{\epsilon, 0, \dots\}$



Language: strings over $\Sigma = \{0,1\}$ with an even number of 1's.

- $Q = \{q_E, q_O\}$
- Start state: q_E
- Accepting states: $F = \{q_E\}$



$$\delta(q_E, 1) = q_O, \quad \delta(q_O, 1) = q_E$$

$$\delta(q_E, 0) = q_E \quad \delta(q_O, 0) = q_O$$

10011
 $q_E \xrightarrow{1} q_O \xrightarrow{0} q_O \xrightarrow{0} q_O \xrightarrow{1} q_E \xrightarrow{1} q_O \Rightarrow \text{rejected}$

(A Quick Note on Notation)

- $f : A \rightarrow B$ means f is a **function** that maps elements of **set** A to **set** B
e.g., $\sin : \mathbb{R} \rightarrow [-1, +1]$

- Suppose C and D are two sets.

Then $C \times D$ ² is the set of all **tuples** (pairs) of the form (c,d) where c is an element from set C and d is an element from set D

Example:

$$C = \{1,2\}$$

$$D = \{x,y,z\}$$

$$C \times D = \{(1,x), (1,y), (1,z), (2,x), (2,y), (2,z)\}$$

- Therefore $\delta : Q \times \Sigma \rightarrow Q$ simply is a function that maps **(state,symbol)** pairs to **states**

i.e., if you are in **state** q and **read symbol** x transition to **state** q'

²Cartesian product of C and D

DFA Representations

Example: strings over $\Sigma = \{0,1\}$ with an **even number of 1's**

set of states $Q = \{q_E, q_O\}$ start state q_E set of accept states $F = \{q_E\}$.

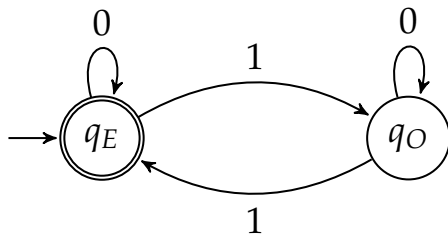
- ① State transition function $\delta : Q \times \Sigma \rightarrow Q$:

$$\delta(q_E, 0) = q_E, \quad \delta(q_E, 1) = q_O, \quad \delta(q_O, 0) = q_O, \quad \delta(q_O, 1) = q_E.$$

- ② State transition table:

		Σ	
		0	1
states	q_E	q_E	q_O
	q_O	q_O	q_E

- ③ Diagram:



Using DFA to Solve (Some) Decision Problems

solving a decision prob. $\Leftrightarrow$ recognizing the corresponding language L ³

- **Input:** A string over an alphabet Σ
- The machine reads symbols **left to right**, one at a time
- After reading the last symbol, the machine must decide:

Accept or Reject the string?

Therefore our goal is to:

design a DFA that accepts input string w if and only if $w \in L$
(rejects it otherwise if $w \notin L$).

³i.e., given an input $w \in \Sigma^*$, decide whether $w \in L$ or $w \notin L$

Designing DFAs: General Strategy

- DFA has no memory other than its current state.
- When designing a DFA, ask yourself:
“What is the minimal amount of information I need to remember about the string read so far to accept/reject the string at the end?”
- Each state corresponds to a specific piece of remembered information.
- You must define a transition for **every** state and **every** symbol in Σ .

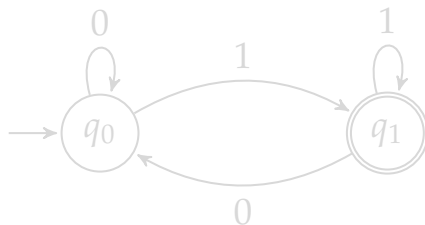
Example 1: Strings Ending in '1'

Build a DFA for the language of strings over $\Sigma = \{0,1\}$ that **end with a 1**

What do we need to remember?

We only care about the *last* symbol we saw.

- State q_0 : The last symbol was 0 (or we haven't read anything yet).
- State q_1 : The last symbol was 1 (this is our accept state!)



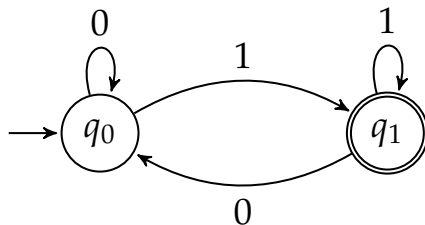
Example 1: Strings Ending in '1'

Build a DFA for the language of strings over $\Sigma = \{0,1\}$ that **end with a 1**

What do we need to remember?

We only care about the *last* symbol we saw.

- State q_0 : The last symbol was 0 (or we haven't read anything yet).
- State q_1 : The last symbol was 1 (this is our accept state!)

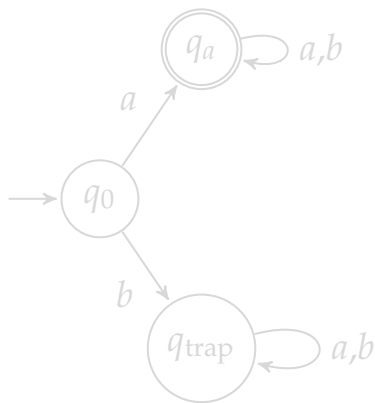


Example 2: The "Trap" State

Build a DFA for strings over $\Sigma = \{a,b\}$ that **start with the letter 'a'**.
What do we need to remember?

We only care about the *very first* symbol.

- State q_0 : Start state (haven't read anything).
- State q_a : The first symbol was a (Accept! We stay here forever).
- State q_{trap} : The first symbol was b (Reject! The string is ruined, and no future input can fix it).



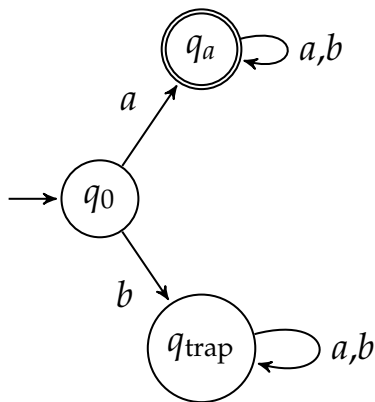
Example 2: The "Trap" State

Build a DFA for strings over $\Sigma = \{a,b\}$ that **start with the letter 'a'**.

What do we need to remember?

We only care about the *very first* symbol.

- State q_0 : Start state (haven't read anything).
- State q_a : The first symbol was a (Accept! We stay here forever).
- State q_{trap} : The first symbol was b (Reject! The string is ruined, and no future input can fix it).



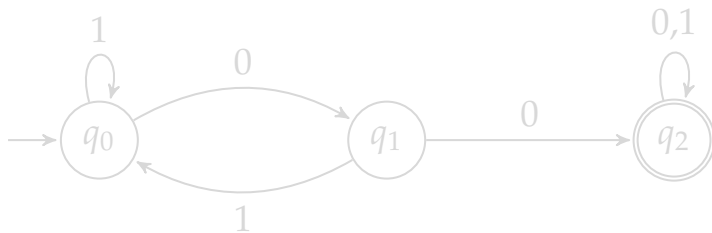
Example 3: Containing a Substring

Build a DFA for strings over $\Sigma = \{0,1\}$ that **contain the substring "00"**.

What do we need to remember?

How far along are we in seeing the pattern "00"?

- State q_0 : Haven't seen a 0 recently.
- State q_1 : Saw one 0 (if we see a 1 now, our progress resets to q_0).
- State q_2 : Saw "00"! (Accept! We stay here forever, because the string now contains "00").



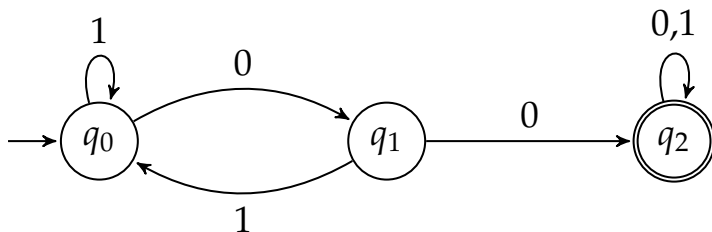
Example 3: Containing a Substring

Build a DFA for strings over $\Sigma = \{0,1\}$ that **contain the substring "00"**.

What do we need to remember?

How far along are we in seeing the pattern "00"?

- State q_0 : Haven't seen a 0 recently.
- State q_1 : Saw one 0 (if we see a 1 now, our progress resets to q_0).
- State q_2 : Saw "00"! (Accept! We stay here forever, because the string now contains "00").



Remarks

- Each state represents where you are in processing the input.
- A DFA must specify exactly one transition for each symbol in Σ from each state.
- If no valid transition is specified, or transitions are incomplete, the machine isn't a proper DFA.